AULA 14

Aterrissagem do TP3: o enunciado, item por item

Etapa 7 · terça, 20/10/2026 · sala SJ205 · Prof. Dácio Moreira de Souza

O enunciado é a rubrica.

Cada verbo dele — execute, capture, explique, publique, relacione — é um item que alguém vai procurar no seu repositório na sexta à noite.

Cenas dos últimos episódios: onde paramos

- A máscara virou instância: 72 objetos onde havia 120, e nenhum erro na tela
- A régua mostrou IoU 0,86 com 48 casadas de 120 — número bonito de medida quebrada
- O maior IoU da tabela era do pior detector: o modelo, com 4 casadas de 120
- Percepção veicular não saiu — e entra hoje, no formato que o enunciado pede

Quatro itens, e o verbo que pega em cada um

- 1. Launch principal — EXECUTE com um único comando, e DOCUMENTE com captura
- 2. TFs e SLAM — CAPTURE o TF tree e EXPLIQUE cada frame no relatório
- 3. Segmentação semântica — PUBLIQUE em /vision/segmented, para ver no RViz2
- 4. Arquitetura veicular — RELACIONE com as escolhas do seu projeto

A versão da disciplina é o Gazebo Fortress.

O enunciado cita Harmonic; o par do Humble é o Fortress, e ninguém perde ponto por entregar com ele. Cada distribuição do ROS 2 é testada contra UMA versão do Gazebo, e os pacotes do Harmonic conflitam com os do Humble: trocar hoje derruba o que funciona. É alinhamento de versão — o mesmo que quebra o cv_bridge com NumPy 2.

Como a aula funciona hoje

- Sem conteúdo novo: o enunciado lido item a item, com cada um rodando na minha tela
- Você reproduz no seu projeto enquanto eu circulo — traga o que não está fechando
- No fim, um roteiro de revisão: ele aponta onde olhar — não corrige e não valida nada
- Censo agora: quem sobe o sistema com um comando? quem tem frames.pdf e mapa salvo?

ITEM 1

O launch principal

Um comando, e o sistema inteiro no ar

```
cd "$PB_WS" && source install/setup.bash
ros2 launch <seu_pacote> bringup.launch.py

# o que tem de subir, na ordem:
#   Gazebo (ou o seu mundo)  ->  Robot State Publisher  ->  RViz2  ->  percepção
```

E a captura de tela faz parte do item: o enunciado pede o launch DOCUMENTADO.

As quatro exigências cabem numa tela

launch/bringup.launch.py

```
IncludeLaunchDescription(              # 1. em sequencia
    PythonLaunchDescriptionSource(mundo_launch),
    launch_arguments={"mundo": mundo}.items()),

Node(package="robot_state_publisher",   # 2. o URDF
     parameters=[{"robot_description": urdf}]),

Node(package="rviz2", executable="rviz2",
     arguments=["-d", os.path.join(pkg, "rviz", "projeto.rviz")]),  # 3. o .rviz

Node(package="meu_pacote", executable="percepcao",
     parameters=[os.path.join(pkg, "config", "sistema.yaml")]),     # 4. o YAML
```

O .rviz e o YAML têm de estar NO REPOSITÓRIO — no clone do avaliador, o que não foi commitado não existe.

O parâmetro chegou mesmo? Dois segundos resolvem

```
ros2 param get /slam_toolbox base_frame

    String value is: base_footprint        # o valor do YAML
```

Se o número ainda estiver no código, é ele que ganha — e o YAML é lido e descartado em silêncio. É a Aula 10 voltando.

As três que travam o item 1, e são sempre as mesmas

- O .rviz fora do repositório: funciona aqui, abre vazio no clone de quem corrige
- O parâmetro ainda no código: o enunciado diz "todos os parâmetros via YAML"
- Dois bringup.launch.py: nem você nem eu sabemos qual o ros2 launch vai pegar
- Nenhuma das três dá erro. As três custam o item inteiro.

ITEM 2

TFs e SLAM

A árvore, e a correção acontecendo

```
cd "$PB_DIR/docs/evidencias/tp3" && ros2 run tf2_tools view_frames
ros2 run tf2_ros tf2_echo map odom

# uma árvore só, map no topo, e números que MUDAM
```

O PDF vazio quer dizer que o launch não estava rodando: TF é fluxo, não arquivo.

O que escrever sobre cada frame — é o que "explique" cobra

- map — publicado pelo SLAM: fixo, e dá SALTOS quando o laço fecha
- odom — de quem integra a odometria: contínuo, suave, e erra acumulando
- base_link — a raiz do seu URDF: o corpo do robô
- laser, camera... — do robot_state_publisher: onde cada sensor está no corpo
- Lista de frames sem dono não é explicação. Diga QUEM PUBLICA cada um.

map → odom é o erro da odometria, com o sinal trocado.

É por isso que essa aresta existe, e por isso quem a publica é o SLAM, e não você. Essa frase, no relatório, é a diferença entre suficiente e bom neste item.

Salvar o mapa — com o launch AINDA no ar

```
mkdir -p "$PB_DIR/maps" && cd "$PB_DIR/maps" && \
  ros2 run nav2_map_server map_saver_cli -f mapa

# gera o par mapa.pgm + mapa.yaml
```

O mapa vive na memória do nó: com o launch fechado, não há o que salvar.

O que eu procuro no item 2, na ordem

- O frames.pdf no repositório — e uma árvore só, sem frame órfão e sem dois odom
- O par mapa.pgm + mapa.yaml, commitado (evidência no disco e fora do git não existe)
- Um parágrafo por frame, dizendo quem publica e o que significa
- E a frase do map → odom: quem a escreve mostrou que entendeu o TP inteiro

ITEM 3

Segmentação semântica com YOLO

São três exigências separadas, e a segunda é a que some

- Máscaras coloridas POR CLASSE — não uma cor por objeto
- Transparência 0,5 — não a máscara sozinha, não o contorno sozinho
- SOBREPOSTAS ao frame — a imagem original tem de aparecer por baixo
- E publicadas em /vision/segmented, para ver no RViz2 — não no rqt_image_view

O exemplo da Aula 13 faz as três

```
cd "$PB_WS" && source install/setup.bash
ros2 launch aula13_instancia instancia.launch.py \
  saida:=classe_alpha alpha:=0.5 rviz:=true

# saida: classe_alpha (o que o TP3 pede) | instancia | mascara
```

Troque as classes do instancia.yaml pelas do SEU domínio — vermelho e azul são as minhas.

Com o YOLO, que é o que o enunciado pede

```
cd "$PB_DIR/aula13-instancia"
uv venv --system-site-packages .venv-yolo
uv pip install --python .venv-yolo/bin/python ultralytics

PYTHONPATH="$PWD/.venv-yolo/lib/python3.10/site-packages:$PYTHONPATH" \
  ros2 launch aula13_instancia instancia.launch.py metodo:=modelo saida:=classe_alpha
```

O --system-site-packages faz o ambiente enxergar o rclpy; o PYTHONPATH devolve o ultralytics ao nó. pip solto troca o NumPy do sistema e derruba o cv_bridge.

A imagem some no RViz2, e o aviso está no outro terminal.

É QoS: display em Reliable contra publicador Best Effort. O RViz2 fica mudo e o status do display continua Ok — mas o NÓ avisa: "requesting incompatible QoS. No messages will be sent to it". Quem reclama é quem não consegue entregar, não quem não recebe. A cura: Reliability Policy = Best Effort.

O YOLO não conhece o seu domínio — e isso tem três saídas

- Ele conhece as 80 classes do COCO: "peça vermelha do meu projeto" não está lá
- 1. Use as classes do COCO que existirem na sua cena, e diga isso
- 2. Deixe o modelo achar a instância e vote a classe pela cor dentro da máscara
- 3. Entregue o clássico e justifique com a tabela da Aula 13 — com número
- As três passam. O que não passa é escolher sem medir.

ITEM 4

Arquitetura de percepção veicular

Dois barramentos, porque resolvem problemas diferentes

- CAN — controle: powertrain, freios, portas. Mensagem curta, 8 bytes (64 no CAN FD)
- Todos escutam, e a prioridade está no identificador. Dois fios, previsível, barato
- Ethernet automotiva — dados: câmera, LiDAR, atualização. 100BASE-T1, um par trançado
- Comutada, pilha IP completa, banda que o sensor exige

O CAN é escolhido por previsibilidade, não por velocidade.

A Ethernet entra onde a banda do sensor não cabe no CAN. Uma câmera de 1080p não passa por mensagens de 8 bytes; um comando de freio não precisa de banda, precisa de chegar na hora. Quem escreve que a Ethernet "substituiu" o CAN não entendeu.

Cada sensor entra no ROS 2 por um caminho — e com um cuidado

- Câmera -> /camera/image_raw (sensor_msgs/Image): QoS sensor data; comprima no bag
- LiDAR -> /scan ou /points: a TF do sensor ao base_link precisa estar certa
- IMU -> /imu/data: alta taxa, e é o que suaviza a odometria
- GNSS -> /fix (NavSatFix): referencial diferente do map, precisa de conversão
- CAN do veículo -> ros2_socketcan -> /from_can_bus (can_msgs/Frame)
- E para todos: carimbo de tempo e TF. Sem os dois, a fusão mistura passado com presente

ANTES DE POSTAR

A revisão e a entrega

O roteiro de revisão aponta onde olhar

```
cd "$PB_DIR" && python3 revisar_tp3.py
cd "$PB_DIR" && python3 revisar_tp3.py --ao-vivo     # com o sistema no ar

    achei      existe UM bringup.launch.py        launch/bringup.launch.py
    nao achei  docs/evidencias/tp3/frames.pdf     nao achei (ou vazio)
    olhe       cada frame explicado no relatorio  leitura do professor
```

Não é correção nem validação: "achei" não quer dizer certo, "não achei" não quer dizer errado. Nos dois casos quer dizer: vá olhar isso.

A entrega de sexta, na ordem

```
cd "$PB_DIR"
git add -A && git commit -m "TP3: evidências e relatório"
git switch -c entrega-tp3 && git push -u origin entrega-tp3
git tag -a tp3 -m "entrega TP3" && git push origin tp3

# e no Moodle: o ZIP, o PDF do relatório e os links
```

Sem a postagem no Moodle não há nota. E tag local, sem push, não aparece para ninguém.

Para a próxima aula, 27/10 — a Etapa 8 abre com navegação

- Ela ficou para trás três vezes, e agora vira entrega: o Nav2 em cima do seu mapa
- O TP4 começa em 30/10, e o primeiro gate é o AMCL convergindo
- Chegue com o TP3 postado e a cabeça livre
- E com o mapa salvo: ele é a base do TP4 inteiro